

BÁO CÁO KỸ THUẬT

Tái hiện & Tối ưu hóa Mô hình Phân vùng Tâm nhĩ Trái với DINOv2

arXiv: 2411.09598v1 | Thời gian: 07–08/2026

Công cụ: PyTorch · DINOv2 · Google Colab T4 GPU

Tài liệu tham khảo: "DINOv2-based Left Atrium Segmentation", arXiv:2411.09598v1

Thời gian thực hiện: 07/2026 – 08/2026

Repository: github.com/TrungNGD195/Left_atrium_segmentation

1. Tổng quan & Công cụ thực hiện

1.1. Nội dung bài báo gốc

Bài báo "DINOv2-based Left Atrium Segmentation" (arXiv:2411.09598v1) đề xuất một pipeline phân vùng tâm nhĩ trái từ ảnh MRI 3D, tận dụng mô hình nền tảng DINOv2 — một Vision Transformer (ViT) được pre-trained theo phương pháp self-supervised trên hơn 142 triệu ảnh, từ đó khai thác khả năng trích xuất đặc trưng tổng quát mà không cần nhãn.

Các thành phần chính bài báo đề xuất:

- Backbone DINOv2 (ViT-Small): Được dùng làm bộ trích xuất đặc trưng cố định (frozen encoder). Ảnh MRI 448×448 được chia thành $32 \times 32 = 1024$ patch, mỗi patch kích thước 14×14 pixel, và được mã hóa thành một vector đặc trưng 384 chiều.
- Lightweight Decoder: Một bộ decoder nhỏ gọn gồm các khối Upsample + Conv kết hợp đặc trưng từ encoder để tái tạo mask phân vùng kích thước đầy đủ (448×448).
- Pipeline tiền xử lý 2D: Ảnh MRI 3D dạng Nifti được cắt thành từng lát cắt 2D theo trục z, chuẩn hóa về kích thước và cường độ pixel trước khi đưa vào mô hình.
- Năm cải tiến dần dần (mỗi cái giải quyết một vấn đề cụ thể của mô hình ban đầu, sẽ được phân tích ở Phần 3 & 4).

1.2. Những gì đã tái hiện trong code ban đầu

Code baseline được xây dựng bám sát kiến trúc lõi mà bài báo mô tả, gồm các thành phần sau:

a) Tiền xử lý dữ liệu (`src/prepare_data.py`):

- Đọc toàn bộ volume MRI 3D định dạng `.nii.gz` bằng NiBabel
- Cắt từng lát cắt 2D theo trục axial (trục z), loại bỏ các lát cắt không chứa tâm nhĩ (toàn nền đen)
- Resize ảnh về kích thước 448×448 pixel — con số được chọn vì là bội số nguyên của `patch_size=14` ($448 = 14 \times 32$)
- Chuẩn hóa theo mean và std của ImageNet vì DINOv2 được pre-trained trên ảnh tự nhiên
- Lưu cặp (`image.png`, `mask.png`) vào thư mục `train_2d/`, `val_2d/`, `test_2d/`

b) Dataset Loader (`src/dataset.py`):

- Class `LeftAtriumDataset` kế thừa `torch.utils.data.Dataset`
- Augmentation cơ bản: random horizontal/vertical flip, normalization
- Trả về tuple (`image`, `mask`, `filename`) với mask nhị phân `{0, 1}`

c) Kiến trúc mô hình (`src/model.py`):

- `DINOv2Encoder`: Tải backbone `dinov2_vits14` qua `torch.hub`, đóng băng toàn bộ tham số (`requires_grad = False`), chỉ thực hiện forward pass để lấy đặc trưng (`B, 384, 32, 32`)
- `SegmentationDecoder`: 3 khối `UpBlock` (`Upsample×2 → Conv 3×3 → BN → ReLU`), upsample từ `32→64→128→256` rồi dùng `Conv 1×1` chiếu về 1 kênh → logit mask
- `DINOv2Segementer`: Wrapper kết hợp encoder + decoder thành một module hoàn chỉnh

d) Vòng lặp huấn luyện (`src/train.py`, `src/train_fast.py`):

- Hàm mất mát ban đầu: `BCEWithLogitsLoss` đơn thuần
- Optimizer: `Adam`, learning rate mặc định `1e-3`
- Lưu checkpoint tốt nhất theo `val_dice`
- Metric đánh giá: Dice Coefficient và IoU (Jaccard Index)

e) Đánh giá (`src/evaluate.py`):

- Chạy inference trên tập test, tính Dice và IoU trên từng lát cắt
- Xuất hình ảnh trực quan hóa: ảnh MRI gốc + ground truth mask + predicted mask đặt cạnh nhau

1.3. Công cụ và môi trường

Ngôn ngữ lập trình:

- Python 3.10 — toàn bộ pipeline huấn luyện và đánh giá

Framework và thư viện cốt lõi:

- PyTorch 2.x — framework học sâu chính
- torchvision — tiền xử lý ảnh và augmentation
- torch.hub — tải mô hình DINOv2 pretrained từ `facebookresearch/dinov2`
- torch.cuda.amp — Automatic Mixed Precision (AMP FP16) để tăng tốc huấn luyện (thêm vào ở giai đoạn tối ưu)

Xử lý dữ liệu y khoa:

- NiBabel — đọc file ảnh MRI định dạng NIFTI (`.nii.gz`)

- NumPy, SciPy — xử lý mảng đa chiều và phép toán hình thái học
- Pillow (PIL) — lưu lát cắt 2D dưới dạng ảnh PNG

Đánh giá và trực quan hóa:

- Matplotlib — vẽ training curves và overlay mask dự đoán
- tqdm — thanh tiến trình huấn luyện

Môi trường thực thi:

- Google Colab (T4 GPU, 15GB VRAM, 25GB RAM) — huấn luyện và đánh giá chính
- Google Drive — lưu trữ dữ liệu, checkpoint và kết quả
- Jupyter Notebook (`notebooks/colab_train.ipynb`) — giao diện tương tác toàn bộ pipeline

Phần cứng:

- GPU: NVIDIA Tesla T4 (15GB GDDR6, 2,560 CUDA Cores, Tensor Cores thế hệ 2)
- Tốc độ huấn luyện ban đầu (CPU): ~45 phút/lần chạy → sau tối ưu (GPU T4 + AMP FP16): ~4 giây/epoch

2. Đánh giá kết quả ban đầu (Baseline)

2.1. Kiến trúc baseline

Mô hình baseline được xây dựng theo đúng kiến trúc bài báo:

- Encoder: DINOv2 ViT-Small (`vit_small`, 21.7M tham số) — đồng bộ hoàn toàn
- Decoder: 3 khối Upsample×2 + Conv + BN + ReLU (32→64→128→256) → Conv 1×1 → Mask
- Hàm mất mát: Binary Cross-Entropy (BCE) đơn thuần

Pipeline xử lý dữ liệu:

1. Ảnh MRI 3D NIfTI (kích thước ~640×640×88 voxel) được cắt thành các lát cắt 2D theo trục z
2. Resize về 448×448 pixels (khớp với `patch_size=14` của DINOv2: $448/14 = 32$ patches)
3. Chuẩn hóa theo mean/std của ImageNet (do DINOv2 pretrained trên ImageNet)

Phân chia dữ liệu:

- Train: 897 lát cắt (từ 80 volume)
- Val: 191 lát cắt (từ 20 volume)
- Test: 263 lát cắt (từ 20 volume)

2.2. Kết quả baseline

Chỉ số	Giá trị trên Val	Giá trị trên Test
Dice Coefficient (Mean)	0.8203	0.7227
IoU / Jaccard (Mean)	0.6994	0.5874

> Ghi chú: Bài báo gốc báo cáo Dice $\sim 0.72\text{--}0.73$ trên tập test với cấu hình baseline tương đương. Kết quả tái hiện của chúng tôi đạt 0.7227, sai lệch chấp nhận được ($\sim 0.5\%$).

3. Phân tích vấn đề (Bottlenecks)

Sau khi đánh giá kết quả baseline, năm vấn đề kỹ thuật được xác định là nguyên nhân chính cản trở độ chính xác:

Vấn đề 1 — Mất cân bằng lớp nặng (Class Imbalance)

Phân tích:

- Tâm nhĩ trái chỉ chiếm $\sim 3\text{--}8\%$ tổng diện tích ảnh, phần còn lại là nền đen và các cấu trúc tim khác.
- Hàm BCE Loss đơn thuần coi mỗi pixel có trọng số như nhau $\rightarrow$ gradient bị chi phối bởi 92–97% pixel nền $\rightarrow$ mô hình học xu hướng dự đoán toàn bộ là nền (trivial solution), đặc biệt ở các lát cắt ngoài biên tâm nhĩ.
- Biểu hiện cụ thể: Dice score trên các lát cắt có tâm nhĩ nhỏ (đầu và cuối volume) gần bằng 0.

Vấn đề 2 — Chênh lệch miền dữ liệu (Domain Shift)

Phân tích:

- DINOv2 được pre-trained trên ImageNet-22K (ảnh tự nhiên: chó, mèo, phong cảnh...). Đặc trưng học được thiên về màu sắc, texture RGB tự nhiên.
- Ảnh MRI là ảnh grayscale 1 kênh được stack thành 3 kênh giống nhau, có cường độ sáng phân bố hoàn toàn khác (HU units, Rician noise).
- Khi đóng băng hoàn toàn encoder, các đặc trưng ViT Attention Head tập trung vào các pattern không liên quan đến cấu trúc tim $\rightarrow$ decoder phải "tự đoán" với thông tin không đặc trưng.
- Biểu hiện: Decoder học chậm, dễ bị overfit sau epoch thứ 10–15 mặc dù encoder vẫn frozen.

Vấn đề 3 — Mất chi tiết đường viền (Coarse Boundary)

Phân tích:

- DINOv2 chia ảnh thành các patch 14×14 pixels. Với ảnh 448×448 , encoder chỉ tạo ra feature map 32×32 (không gian thông tin bị nén $196\times$ so với ảnh gốc).
- Decoder gốc chỉ dùng đặc trưng từ layer cuối cùng (layer 12) — layer sâu nhất của ViT — chứa thông tin ngữ nghĩa cao nhưng mất gần như toàn bộ thông tin không gian cục bộ (edges, local texture).
- Khi upsample từ $32 \rightarrow 448$, đường viền tâm nhĩ bị vuông vức theo lưới patch 14×14 , giống hiện tượng "blocky artifacts" trong JPEG nén mạnh.
- Biểu hiện: Dice Mean cao nhưng Dice Max chỉ đạt ~ 0.94 (không đạt $0.97+$ như ảnh tốt nhất cần có), mô hình không học được đường cong mượt của thành tâm nhĩ.

Vấn đề 4 — Nhiễu đốm rời rạc (Spurious False Positives)

Phân tích:

- Sau khi sigmoid + threshold (>0.5), dự đoán có nhiều điểm/cụm pixel dương tính sai nằm rải rác bên ngoài vùng tâm nhĩ thực sự.
- Nguyên nhân: một số lát cắt MRI có cường độ sáng tương đồng với tâm nhĩ ở các vùng khác (van tim, động mạch chủ, mỡ tim), và mô hình không có cơ chế nhận biết sự liên thông không gian.
- Biểu hiện: IoU bị kéo giảm đáng kể do diện tích dự đoán sai (phần tử của union tăng).

Vấn đề 5 — Thiếu ngữ cảnh không gian 3D

Phân tích:

- Tâm nhĩ là một cấu trúc ba chiều liên tục trong không gian. Nếu chỉ xét từng lát cắt 2D độc lập, mô hình không biết lát cắt hiện tại đang ở vị trí nào trong volume (đầu, giữa hay cuối tâm nhĩ).
- Các lát cắt nằm ở biên trên/dưới của tâm nhĩ (nơi tâm nhĩ mới xuất hiện hoặc sắp kết thúc) rất khó phân biệt với lát cắt không có tâm nhĩ vì hình dạng nhỏ và mờ.
- Biểu hiện: Sai lệch lớn giữa Dice Min (~ 0.0) và Dice Mean (~ 0.73) $\rightarrow$ phương sai cao $\rightarrow$ mô hình không ổn định.

4. Giải pháp Tối ưu hóa

Giải pháp 1 — BCE + Dice Loss kết hợp

Kỹ thuật áp dụng: Compound Loss Function

Cơ chế hoạt động:

$$\mathcal{L}_{\text{total}} = 0.5 \cdot \mathcal{L}_{\text{BCE}} + 0.5 \cdot \mathcal{L}_{\text{Dice}}$$

- BCE Loss (`BCEWithLogitsLoss`): tối ưu từng pixel riêng lẻ, giúp gradient ổn định.
- Dice Loss ($1 - \text{Dice}$): tối ưu tỷ lệ chồng lấp giữa predicted mask và ground truth, hoàn toàn không bị ảnh hưởng bởi class imbalance vì công thức chỉ tính trên vùng dương tính.

```
dice = (2 * (probs * targets).sum(dim=(2,3)) + smooth) / \
        (probs.sum(dim=(2,3)) + targets.sum(dim=(2,3)) + smooth)
dice_loss = 1.0 - dice.mean()
```

Kết quả: Val Dice 0.8203 $\rightarrow$ 0.8343 (+1.4%), Test Dice 0.7227 $\rightarrow$ 0.7376 (+2.1%).

Giải pháp 2 — LoRA Fine-tuning Encoder

Kỹ thuật áp dụng: Low-Rank Adaptation (LoRA) + Differential Learning Rate

Cơ chế hoạt động:

Thay vì unfreeze hoàn toàn encoder (rất tốn VRAM và dễ overfit), LoRA chèn thêm 2 ma trận hạng thấp vào trọng số Attention của N block cuối:

$$W_{\text{new}} = W_{\text{frozen}} + \Delta W = W_{\text{frozen}} + B \cdot A \cdot \frac{\alpha}{r}$$

Trong đó:

- $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$ — ma trận LoRA (rank $r=4$)
- W_{frozen} — trọng số gốc của DINOv2, không thay đổi (gradient=False)
- Chỉ train A và B → chỉ thêm ~0.5% tham số so với toàn bộ encoder

Differential Learning Rate:

Decoder parameters:	$lr = 1e-3$	(học nhanh - task-specific)
LoRA parameters:	$lr = 1e-5$	(học chậm - 100× nhỏ hơn, tránh phá vỡ pretrained knowledge)

Tối ưu hóa tốc độ bổ sung: AMP FP16 (Automatic Mixed Precision) — tận dụng Tensor Cores của GPU T4, giảm thời gian huấn luyện từ ~45 phút (CPU) xuống còn ~2–3 phút (GPU T4 với batch_size=32).

Bug đã gặp và sửa: Sau khi `inject_lora_into_backbone()`, các tham số LoRA mới sinh ra nằm trên CPU trong khi mô hình đã được `.to(device)` trước đó → `RuntimeError: Expected all tensors to be on same device`. Fix: Gọi thêm `model = model.to(device)` ngay sau khi inject.

Kết quả: Val Dice 0.8343 → 0.8494 (+1.5%), Test Dice 0.7376 → 0.7819 (+5.7%).

Giải pháp 3 — FPN Multi-scale Decoder

Kỹ thuật áp dụng: Feature Pyramid Network (FPN) thích nghi cho Vision Transformer

Cơ chế hoạt động:

Thay vì chỉ lấy đặc trưng từ layer cuối, dùng API `backbone.get_intermediate_layers(x, n=4)` của DINOv2 để trích xuất đặc trưng từ 4 layer cuối (lớp 9, 10, 11, 12):

```

Layer 9: Đặc trưng "nông" → giữ thông tin cạnh, texture cục bộ
Layer 10: Đặc trưng trung gian
Layer 11: Đặc trưng trung gian
Layer 12: Đặc trưng "sâu" → ngữ nghĩa cao, biết "đây là tâm nhĩ"
    
```

Pipeline FPN:

1. Lateral Projection: Mỗi layer: 384 channels → 128 channels (Conv 1×1 + BN + ReLU)
2. Top-down Merge: Cộng đặc trưng deep vào shallow (do cùng kích thước 32×32): `merged[i] = fpn[i] + merged[i+1]`
3. Smooth Conv: Conv 3×3 sau mỗi merge để làm mịn
4. Concatenate: $4 \times 128 = 512$ channels → Upsample 32→64→128→256→448 → Mask

Kết quả: Val Dice 0.8343 (baseline) → 0.8784 (+4.4%), Test Dice 0.7376 → 0.8151 (+10.5%) — kết quả tốt nhất trong toàn bộ thực nghiệm.

Giải pháp 4 — Hậu xử lý LCC

Kỹ thuật áp dụng: Largest Connected Component (LCC) — Morphological Post-processing

Cơ chế hoạt động:

```
from scipy import ndimage

labeled, num_features = ndimage.label(pred_binary)
component_sizes = ndimage.sum(pred_binary, labeled, range(1, num_features + 1))
largest_label = np.argmax(component_sizes) + 1
lcc_mask = (labeled == largest_label).astype(np.float32)
```

Tâm nhĩ là một cấu trúc liên thông duy nhất trong không gian → sau khi nhấn hóa các vùng liên thông (connected components), chỉ giữ lại vùng lớn nhất và xóa tất cả các đốm nhiễu còn lại.

Lưu ý: Phương pháp này là post-processing thuần túy, không cần huấn luyện lại. Áp dụng trực tiếp sau khi inference trên kết quả thô của bất kỳ mô hình nào.

Kết quả: Test Dice 0.7376 → 0.7311 (giảm nhẹ -0.6%). Kết quả này cho thấy mô hình gốc (Mục 1) có ít false positives, nên LCC loại bỏ một số vùng dương thực (true positives) nhỏ ở biên tâm nhĩ.

Giải pháp 5 — Đầu vào 2.5D

Kỹ thuật áp dụng: Pseudo-3D (2.5D) Input — Multi-slice Stacking

Cơ chế hoạt động:

Thay vì 1 lát cắt 2D → stack 3 lát cắt liên tiếp vào 3 kênh (R, G, B):

```
Input: [lát z-1, lát z, lát z+1] → tensor (B, 3, 448, 448)
```

- DINOv2 nhận input 3 kênh RGB → hoàn toàn tương thích mà không cần thay đổi kiến trúc encoder.
- Mô hình học được sự biến đổi hình dạng tâm nhĩ giữa các lát cắt liên kề → biết lát cắt hiện tại đang ở đâu trong không gian 3D.
- Với lát cắt biên ($z=0$ hoặc $z=\max$), sử dụng kỹ thuật padding phản chiếu (reflect padding) hoặc lặp lại lát cắt đầu/cuối (edge padding).

Kết quả: Val Dice 0.7965, Test Dice 0.7102 — thấp hơn baseline 2D (0.7376). Nguyên nhân: dataset tương đối nhỏ (897 ảnh train), kỹ thuật 2.5D tăng độ phức tạp đầu vào nhưng không đủ data để học hiệu quả mối tương quan liên lát cắt.

5. Kết luận & Hướng phát triển

5.1. Tổng kết kết quả

Phương pháp	Val Dice	Test Dice	Δ vs Baseline
Baseline (BCE Loss)	0.8203	0.7227	—
Mục 1: BCE + Dice Loss	0.8343	0.7376	+2.1%
Mục 2: LoRA Fine-tuning	0.8494	0.7819	+8.2%

Phương pháp	Val Dice	Test Dice	Δ vs Baseline
Mục 3: FPN Decoder	0.8784	0.8151	+12.8%
Mục 4: LCC Post-processing	—	0.7311	-0.9%
Mục 5: 2.5D Input	0.7965	0.7102	-1.7%

Nhận xét tổng quan:

- FPN Multi-scale Decoder (Mục 3) là giải pháp hiệu quả nhất, đạt Test Dice = 0.8151, cải thiện +12.8% so với baseline. Đây là bằng chứng rõ ràng rằng việc tận dụng đặc trưng đa tầng của ViT là chìa khóa để cải thiện chất lượng phân vùng y khoa.
- LoRA Fine-tuning (Mục 2) là giải pháp hiệu quả thứ hai (+8.2%), đặc biệt phù hợp khi cần thích nghi mô hình pretrained với domain mới mà không cần nhiều tài nguyên tính toán.
- LCC và 2.5D không cải thiện kết quả trên dataset này, cho thấy sự phụ thuộc vào đặc thù dữ liệu.

5.2. Hướng phát triển tiếp theo

Ngắn hạn (trong phạm vi đồ án):

- ☐ Kết hợp LoRA + FPN: Thử nghiệm DINOv2FPNSegmenter với LoRA injected vào encoder để tận dụng cả hai ưu điểm — kỳ vọng Test Dice > 0.83.
- ☐ Tăng cường dữ liệu (Data Augmentation): Thêm random flip, rotation, elastic deformation, intensity jitter cho tập train để tăng tính tổng quát.
- ☐ Thử nghiệm 2.5D + FPN: Kết hợp đầu vào 2.5D với FPN decoder để xem liệu ngữ cảnh 3D có hỗ trợ cho FPN hay không.

Dài hạn (nghiên cứu tiếp theo):

- ☐ Đánh giá 3D (3D Dice): Hiện tại chỉ đánh giá 2D per-slice. Cần tái tạo volume từ các lát cắt và tính Dice/Hausdorff Distance trên toàn volume 3D để có con số so sánh chuẩn với các paper khác.
- ☐ Thử nghiệm với ViT-Base/Large: ViT-Small (embed_dim=384) có thể là bottleneck. ViT-Base (embed_dim=768) với FPN decoder kỳ vọng cải thiện thêm 2–3%.
- ☐ Segment Anything Model (SAM 2): Thay thế DINOv2 bằng SAM 2 encoder (được pre-trained trên dữ liệu đa dạng hơn) để xem khả năng tổng quát hóa sang domain y khoa.
- ☐ Triển khai production: Đóng gói thành REST API (FastAPI + ONNX Runtime) để có thể tích hợp vào hệ thống PACS bệnh viện.